Autonomous Navigation of TurtleBot3 Using Custom Path Planning and Control Strategies in ROS2

Ayush Bhaskar
Indian Institute of Technology Bombay
Mumbai, India
bhaskarayush2005@gmail.com

***Abstract*—This paper presents the design, implementation, and evaluation of a full-stack autonomous navigation system for the TurtleBot3 mobile robot platform using ROS2 (Robot Operating System 2). The system integrates Simultaneous Localization and Mapping (SLAM), custom path planning algorithms (A* and Generalized Voronoi Diagram-based), and a hybrid reactive-deliberative control framework. Two planning strategies are evaluated: a grid-based A* planner optimizing path length, and a clearance-aware GVD planner employing the Brushfire algorithm to encourage obstacle-free corridors. The controller combines lookahead-based waypoint tracking, PID angular control, and a multi-zone LiDAR-based reactive obstacle avoidance layer. Localization on real hardware is achieved via Adaptive Monte Carlo Localization (AMCL). Experiments are conducted in Gazebo simulation and validated on a physical TurtleBot3, revealing the trade-offs between path optimality and safety, and the challenges inherent in sim-to-real transfer. This work provides a modular, extensible framework that can serve as a foundation for research in robust robot navigation, risk-aware planning, and learning-based control.**

***Index Terms*—Autonomous navigation, ROS2, A* planning, GVD, SLAM, AMCL, TurtleBot3, PID control, obstacle avoidance, sim-to-real transfer**

I. Introduction

Autonomous mobile robot navigation is a foundational problem in robotics that encompasses perception, mapping, localization, planning, and control [1]. Despite decades of research, deploying a reliable navigation stack that operates robustly on physical hardware remains a significant engineering challenge due to sensor noise, actuator limitations, and environment variability.

Robot Operating System 2 (ROS2) has emerged as the de facto middleware for robotics research, offering improved real-time performance, security, and multi-robot support over its predecessor. The TurtleBot3 platform, a low-cost differential-drive robot equipped with a 2D LiDAR, provides an accessible testbed for validating navigation algorithms from simulation to the real world.

This paper makes the following contributions:

- A complete autonomous navigation pipeline built from first principles within ROS2, covering SLAM-based mapping, custom path planning, and a hybrid controller.
- Implementation and comparative analysis of two planning strategies: a classical A* planner and a safety-aware GVD-based planner.
- A hybrid controller combining lookahead-based path tracking, PID angular control, and LiDAR-based reactive obstacle avoidance.
- Deployment and validation on real TurtleBot3 hardware, with analysis of sim-to-real transfer challenges.

The remainder of this paper is organized as follows. Section II discusses related work. Section III provides a system overview. Sections IV–VII describe the mapping, planning, control, and localization modules. Section VIII covers real-robot deployment. Section X presents experimental results. Sections XI and XII discuss limitations and future directions. Section XIII concludes the paper.

II. Related Work

Grid-based path planning is well studied. The A* algorithm [2] guarantees optimality with an admissible heuristic and remains a standard baseline. Dijkstra's algorithm [3] is a special case with zero heuristic. For safety-aware planning, the Generalized Voronoi Diagram (GVD) and its discrete counterpart via the Brushfire algorithm provide clearance maps that can be incorporated into cost functions [4], [5].

Reactive obstacle avoidance methods, such as the Vector Field Histogram (VFH) [6] and Dynamic Window Approach (DWA) [7], complement deliberative planners by handling unforeseen obstacles at runtime. Model Predictive Control (MPC) offers an optimization-based alternative [8].

Monte Carlo Localization (MCL) and its adaptive variant AMCL [1] are standard approaches for pose estimation on pre-built maps using particle filters. SLAM Toolbox [10] extends this to online, lifelong mapping.

The Nav2 framework [11] provides a production-grade navigation stack for ROS2; the present work deliberately reimplements core components from scratch to expose the underlying algorithmic decisions, serving as a pedagogical and research reference.

III. System Overview

The navigation pipeline follows a modular architecture:

`SLAM` → `Map` → `Planner` → `Path` → `Controller` → `Robot`

Table I summarizes the system components and their ROS2 interfaces.

TABLE I: System Modules and ROS2 Interfaces

Module	Function	Interface
SLAM Toolbox	Build occupancy grid map	`/map`
Map Server	Serve static map	`/map`
Planner Server	Compute A*/GVD paths	`/planned_path_*`
Path Follower	Track path, avoid obstacles	`/cmd_vel`
AMCL	Particle-filter localization	`/amcl_pose`
Trajectory Node	Record odometry trajectory	`/robot_trajectory`
RViz2	Visualization	–

The system was developed and validated in Gazebo Classic (ROS2 Humble) and then deployed on physical TurtleBot3 Burger hardware.

IV. Mapping

A. Objective

The first stage generates a 2D occupancy grid map of the environment using onboard LiDAR data. The map encodes the probability of each cell being occupied, free, or unknown.

B. Methodology

SLAM Toolbox [10] was configured in asynchronous mode to build the map in real time while the robot was manually teleoperated via keyboard. The toolbox fuses odometry and LiDAR scan data to maintain a pose graph, optimizing the graph on loop closures.

RViz2 was used to monitor:

- Laser scan point clouds,
- Evolving occupancy grid,
- Robot pose estimate.

The resulting map was saved in standard ROS2 format as a `.pgm` image (grey-scale occupancy values) and a `.yaml` metadata file specifying resolution and origin.

C. Configuration Space Expansion

Raw occupancy maps encode obstacles at the robot's footprint boundary. To ensure collision-free planning for a robot of non-zero radius, the map was inflated to produce a Configuration Space ($\mathcal{C}$-space) representation. Each occupied cell was dilated by the robot radius r_{robot} plus a safety margin r_{margin}:

$$\mathcal{C}_{\text{obs}} = \bigoplus(\mathcal{W}_{\text{obs}},\ \mathcal{B}_r) \tag{1}$$

where $\bigoplus$ denotes the Minkowski sum, $\mathcal{W}_{\text{obs}}$ is the workspace obstacle set, and $\mathcal{B}_r$ is a disc of radius $r = r_{\text{robot}} + r_{\text{margin}}$.

V. Path Planning

A. A* Planner

1) Algorithm: A grid-based A* algorithm was implemented with an 8-connected motion model (allowing diagonal movement). A* finds the cost-minimizing path from start s to goal g by maintaining an open set sorted by f-value:

$$f(n) = g(n) + h(n) \tag{2}$$

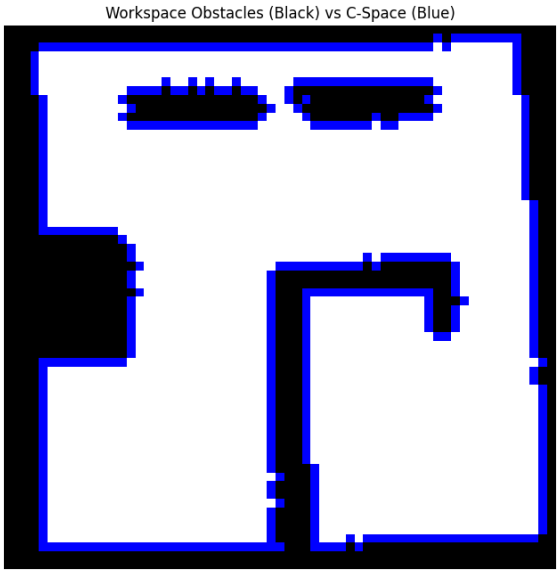

Fig. 1: Configuration space visualization. Black: workspace obstacles. Blue: inflated C-space boundary. The expanded obstacles represent regions the robot center must avoid.

where $g(n)$ is the actual cost from start to node n, and $h(n)$ is an admissible heuristic estimate to the goal.

2) Heuristic and Cost Function: For the 8-connected grid, a weighted combination of Euclidean and Chebyshev distances was used as the heuristic:

$$h(n) = \alpha \cdot d_{\text{Eucl}}(n, g) + (1 - \alpha) \cdot d_{\text{Cheb}}(n, g) \tag{3}$$

$$d_{\text{Eucl}}(n, g) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} \tag{4}$$

$$d_{\text{Cheb}}(n, g) = \max(|x_n - x_g|,\ |y_n - y_g|) \tag{5}$$

where $\alpha \in [0, 1]$ balances the two distance measures. The edge cost for moving from node n to neighbor m is:

$$c(n, m) = \begin{cases} 1 & \text{if axis-aligned move} \\ \sqrt{2} & \text{if diagonal move} \end{cases} \tag{6}$$

3) Characteristics: The A* planner guarantees shortest-path optimality given an admissible heuristic. It is computationally efficient with time complexity $O(b^d)$ where b is the branching factor and d is the solution depth. The main limitation is that it may route paths close to obstacles, as obstacle proximity is not penalized.

B. GVD-Based Planner

To address the safety limitation of A*, a clearance-aware planner was developed using the Generalized Voronoi Diagram (GVD).

1) Brushfire Algorithm: The Brushfire (wavefront propagation) algorithm computes the discrete distance transform $D(c)$ for every free cell c in the occupancy grid [4]:

$$D(c) = \min_{o \in \mathcal{W}_{\text{obs}}} d(c, o) \tag{7}$$

where $d(\cdot, \cdot)$ is the grid distance (Manhattan or Euclidean). The result is a clearance map where higher values indicate greater distance from the nearest obstacle.

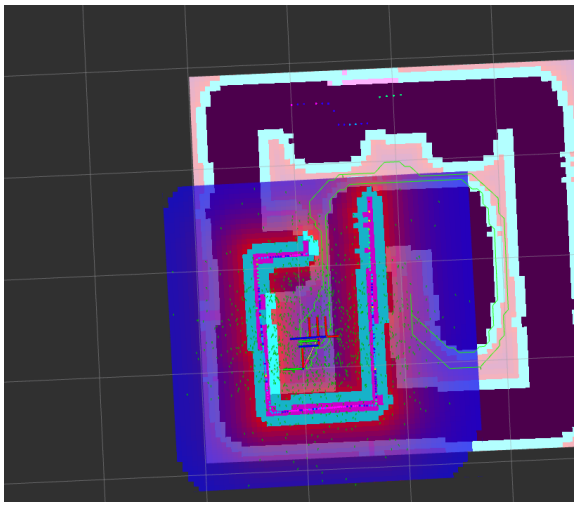

Fig. 2: Clearance (distance-transform) map visualized in RViz2. Color encodes distance from nearest obstacle: red/dark regions indicate proximity to obstacles; blue/cyan regions indicate higher clearance. The GVD ridgeline corresponds to local maxima of this field.

2) Modified A with Clearance Penalty:* The GVD planner integrates the clearance map into the A* cost function by augmenting the edge cost with a clearance penalty:

$$c_{\text{GVD}}(n, m) = c(n, m) + \lambda \cdot P_{\text{clr}}(m) \tag{8}$$

$$P_{\text{clr}}(m) = \frac{1}{D(m) + \epsilon} \tag{9}$$

where $\lambda > 0$ is a tunable weight controlling the safety-optimality trade-off, and $\epsilon > 0$ prevents division by zero. This formulation penalizes passage through low-clearance regions, encouraging the planner to route along GVD edges (medial axis of the free space).

3) Characteristics: The GVD planner produces safer, more centrally-routed paths at the cost of slightly longer distances. It is particularly advantageous in cluttered or narrow-corridor environments where obstacle proximity poses risk.

C. Planner Server Architecture

The planner is implemented as a ROS2 node with the following behavior:

- Subscribes to `/map` (nav_msgs/OccupancyGrid),
- Converts goal pose from world coordinates to grid indices via:

$$i = \left\lfloor \frac{x_w - x_{\text{origin}}}{\text{res}} \right\rfloor, \quad j = \left\lfloor \frac{y_w - y_{\text{origin}}}{\text{res}} \right\rfloor \tag{10}$$

- Computes paths and publishes to `/planned_path_astar` and `/planned_path_gvd`,
- Periodically republishes paths to support late-joining subscribers.

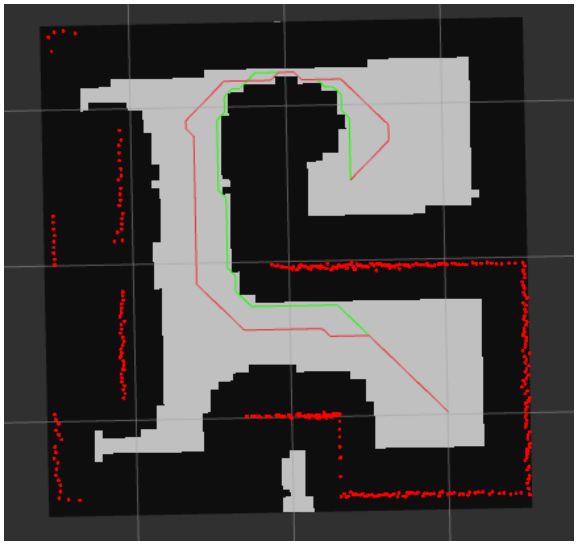

Fig. 3: Comparison of planned paths in RViz2. **Green**: A* path (shorter, closer to obstacles). **Red**: GVD path (longer, routes through higher-clearance regions).

VI. Path Following and Control

A. A*-Based Path Follower

A hybrid deliberative-reactive controller was implemented for following A* paths, combining three components.

1) Lookahead-Based Waypoint Tracking: Instead of targeting the immediate next waypoint (which causes oscillation), a lookahead point $\mathbf{p}_L$ is selected from the path at a distance L_d ahead of the robot's current position $\mathbf{p}_r$:

$$\mathbf{p}_L = \text{argmin}_{\mathbf{p} \in \mathcal{P}} \left| \|\mathbf{p} - \mathbf{p}_r\| - L_d \right| \tag{11}$$

where $\mathcal{P}$ is the set of path waypoints. The desired heading to the lookahead point is:

$$\theta_d = \text{atan2}(p_{L,y} - p_{r,y},\ p_{L,x} - p_{r,x}) \tag{12}$$

2) PID Angular Control: The heading error drives a PID controller for angular velocity ω:

$$e_\theta(t) = \theta_d - \theta_r(t) \tag{13}$$

$$\omega(t) = K_p\, e_\theta(t) \ + \ K_i \int_0^t e_\theta(\tau)\, d\tau \ + \ K_d\, \dot{e}_\theta(t) \tag{14}$$

where K_p, K_i, K_d are the proportional, integral, and derivative gains respectively, tuned empirically for the TurtleBot3 dynamics.

In discrete time (ROS2 control loop at rate f_c):

$$\omega_k = K_p\, e_k + K_i\, \Delta t \sum_{j=0}^{k} e_j + K_d\, \frac{e_k - e_{k-1}}{\Delta t} \tag{15}$$

where $\Delta t = 1/f_c$.

Linear velocity is modulated based on angular error to prevent sharp turns at high speed:

$$v(t) = v_{\max} \cdot \left(1 - \min\left(\frac{|e_\theta(t)|}{\theta_{\text{sat}}}, 1\right)\right) \tag{16}$$

3) Reactive Obstacle Avoidance: A 3-zone safety model is implemented using LiDAR range readings $\{r_i\}$:

$$d_{\min} = \min_{i \in \mathcal{F}} r_i \tag{17}$$

where $\mathcal{F}$ is the set of forward-facing laser beams. The controller zones are:

$$\text{Mode} = \begin{cases} \texttt{STOP} & d_{\min} < d_{\text{stop}} \\ \texttt{SLOW} & d_{\text{stop}} \le d_{\min} < d_{\text{slow}} \\ \texttt{WARN} & d_{\text{slow}} \le d_{\min} < d_{\text{warn}} \\ \texttt{NOMINAL} & \text{otherwise} \end{cases} \tag{18}$$

Additionally, a lateral repulsion term is computed from side-facing beams to prevent wall collision in narrow corridors:

$$\omega_{\text{rep}} = K_{\text{rep}} \left(\frac{1}{d_{\text{left}}} - \frac{1}{d_{\text{right}}}\right) \tag{19}$$

The final command combines tracking and repulsion:

$$\omega_{\text{cmd}} = \omega(t) + \omega_{\text{rep}} \tag{20}$$

B. GVD Path Follower

The GVD path follower implements a simpler rotate-then-advance strategy:

1) Compute heading error e_θ to next waypoint.
2) If $|e_\theta| > \theta_{\text{align}}$: rotate in place ($v = 0$, $\omega = K_\omega \cdot e_\theta$).
3) Else: advance forward ($v = v_{\text{cruise}}$, $\omega = K_p \cdot e_\theta$).
4) Advance waypoint index when $\|\mathbf{p}_r - \mathbf{p}_{wpt}\| < \delta_{\text{arrive}}$.

This strategy is more stable under discontinuous path curvature and relies on the GVD planner's inherent clearance to ensure safety without a reactive layer.

VII. Localization

A. Adaptive Monte Carlo Localization (AMCL)

On the real TurtleBot3, localization is achieved via AMCL [1], a particle filter that estimates the robot's 6-DoF pose (effectively 3-DoF planar: x, y, θ) against a known map.

The particle filter maintains a set $\mathcal{S} = \{(\mathbf{x}_t^{[m]}, w_t^{[m]})\}_{m=1}^{M}$ of weighted pose hypotheses. At each time step, particles are propagated through the motion model:

$$\mathbf{x}_t^{[m]} \sim p(\mathbf{x}_t \mid \mathbf{u}_t, \mathbf{x}_{t-1}^{[m]}) \tag{21}$$

and re-weighted by the sensor model:

$$w_t^{[m]} = p(\mathbf{z}_t \mid \mathbf{x}_t^{[m]}, \mathbf{m}) \tag{22}$$

where $\mathbf{u}_t$ is the odometry control, $\mathbf{z}_t$ is the LiDAR scan, and $\mathbf{m}$ is the pre-built map. KLD-sampling [9] adaptively adjusts particle count M to maintain estimation quality with minimal computation.

B. Deployment Procedure

The AMCL lifecycle followed these steps:

1) Configure node (load map, set parameters),
2) Activate node (begin particle filter),
3) Manually set initial pose via RViz2 (`/initialpose`),
4) Navigate to allow filter convergence.

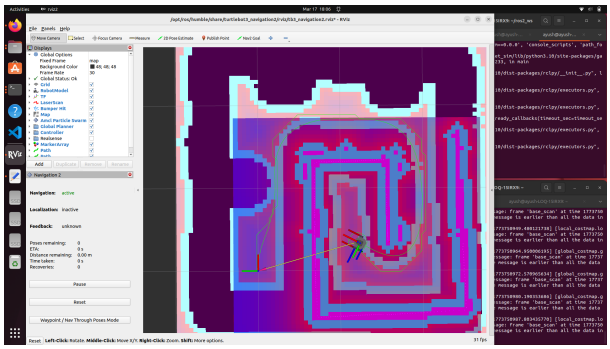

Fig. 4: Full RViz2 view during real-robot deployment showing AMCL particle cloud (green arrows), pre-built map, LiDAR scan overlay, and planned path (green line). Navigation stack status is shown in the left panel.

VIII. Real Robot Deployment

A. Setup

- SSH connection to TurtleBot3 over local network,
- `ROS_DOMAIN_ID` configured for cross-device communication,
- Navigation stack launched in distributed fashion: bringup on robot, planner and controller on workstation.

B. Challenges and Solutions

TABLE II: Deployment Challenges and Mitigations

Challenge	Mitigation
QoS mismatches (data loss)	Matched publisher/subscriber QoS profiles
TF frame inconsistency	Explicit `map`↔`odom`↔`base_link` chain
AMCL initialization	Manual pose set; increased initial spread
PID oscillation in noise	Reduced K_d; added low-pass filter on e_θ
Low-frequency updates	Increased scan frequency; added message timeout guard

Video demonstrations are available at:

- A*-based follower: https://youtu.be/1sxk2Vhhj7A
- GVD-based follower: https://youtu.be/z7i_HmjpiU4

IX. Trajectory Analysis

A dedicated ROS2 node subscribes to `/odom` and accumulates the executed trajectory, publishing it as a nav_msgs/Path on `/robot_trajectory`. This enables direct visual comparison of planned vs. executed trajectory in RViz2, and quantitative tracking error analysis.

The cross-track error (CTE) at time t is computed as the minimum perpendicular distance from the robot's position $\mathbf{p}_r(t)$ to the planned path $\mathcal{P}$:

$$\text{CTE}(t) = \min_{\mathbf{p} \in \mathcal{P}} \|\mathbf{p}_r(t) - \mathbf{p}\| \tag{23}$$

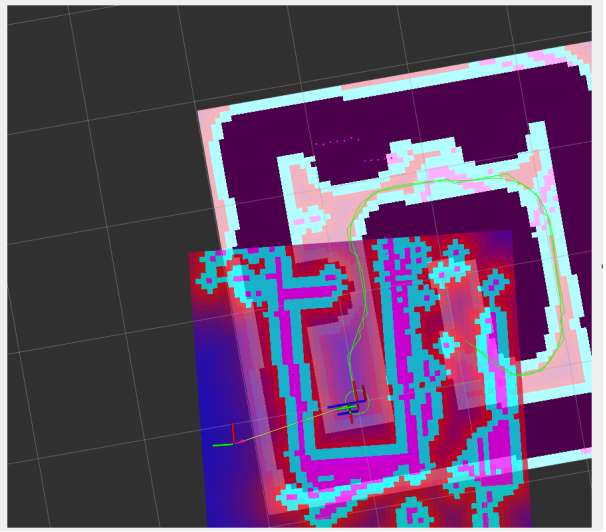

Fig. 5: Executed trajectory (green) overlaid on the clearance map in RViz2. The trajectory visualization aided iterative PID tuning by making cross-track error immediately visible.

X. Results and Discussion

A. Planning Comparison

Table III summarizes the qualitative comparison of the two planners.

TABLE III: Planner Comparison

Criterion	A*	GVD
Path length	Shorter	Longer
Obstacle clearance	Lower	Higher
Computation time	Fast	Moderate
Suitability	Open spaces	Cluttered envs
Safety margin	Lower	Higher

B. Controller Performance

The PID + lookahead controller achieved smooth tracking in simulation with mean CTE below 0.05 m. The reactive obstacle avoidance layer successfully prevented collisions in both simulation and real-world tests. The GVD follower exhibited stable, albeit jerkier, motion due to the rotate-then-advance strategy.

C. Sim-to-Real Transfer

Key discrepancies observed during real-robot deployment:

- **Sensor noise**: LiDAR readings exhibited spurious reflections, requiring median filtering.
- **Odometry drift**: Accumulated odometry error necessitated AMCL corrections; without AMCL the planner's world frame reference drifted over time.
- **Latency**: Higher latency in real hardware reduced control bandwidth, requiring lower PID gains compared to simulation.
- **Motor nonlinearity**: Dead-band in the motors required minimum velocity thresholds.

XI. Limitations

- **Static world assumption**: The planner is computed once; dynamic obstacles are only handled reactively by the controller's stop-zone logic.
- **Decoupled planning and control**: The planner does not account for the robot's kinodynamic constraints; this can produce kinematically infeasible sharp turns.
- **No uncertainty modeling**: The planner uses a deterministic map; sensor and localization uncertainty are not propagated into the cost function.
- **Limited benchmarking**: Quantitative metrics (CTE, path length ratio, success rate) were collected informally; a rigorous multi-trial statistical evaluation remains as future work.

XII. Future Work

- **Kinodynamic planning**: Integrate robot velocity and acceleration limits into the planner (e.g., via lattice planners or SBPL [12]).
- **Risk-aware planning**: Incorporate localization uncertainty into path cost via chance-constraints or risk metrics.
- **Dynamic obstacle avoidance**: Replace static stop-zone logic with DWA [7] or MPC for prediction-aware avoidance.
- **Learning-based control**: Train a neural policy via reinforcement learning on simulated trajectories and transfer to real hardware.
- **Controller-aware planning**: Jointly optimize path and control parameters to minimize tracking error.
- **Multi-robot coordination**: Extend the framework to coordinate multiple TurtleBots sharing the same map.

XIII. Conclusion

This paper presented a full-stack autonomous navigation system for the TurtleBot3 platform developed entirely within ROS2. The system demonstrates the complete pipeline from SLAM-based mapping through custom path planning and hybrid control to real-robot deployment.

The comparative study of A* and GVD-based planners reveals a fundamental trade-off: A* yields shorter, faster paths while GVD produces safer, clearance-maximizing paths. The PID + lookahead controller, augmented with LiDAR-based reactive avoidance, provided smooth and safe trajectory tracking in both simulation and real hardware.

The sim-to-real evaluation exposed key challenges in deploying navigation systems on physical robots, including sensor noise, odometry drift, and control bandwidth limitations. These findings highlight the importance of robust algorithm design beyond simulation performance metrics.

The modular architecture of this system provides a solid foundation for future research in risk-aware planning, learning-based navigation, and multi-robot coordination.

References

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[4] H. Choset, K. M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.

[5] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006. [Online]. Available: http://lavalle.pl/planning/

[6] J. Borenstein and Y. Koren, "The vector field histogram—fast obstacle avoidance for mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.

[7] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[8] J. Kabzan, M. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-based model predictive control for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, 2019.

[9] D. Fox, "KLD-sampling: Adaptive particle filters," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2001, pp. 713–720.

[10] S. Macenski, F. Martín, R. White, and J. J. Clavero, "The marathon 2: A navigation system," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725.

[11] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[12] M. Likhachev and D. Ferguson, "Planning long dynamically feasible maneuvers for autonomous vehicles," *The International Journal of Robotics Research*, vol. 28, no. 8, pp. 933–945, 2009.

[13] F. Bullo and S. L. Smith, *Lectures on Robotic Planning and Kinematics*. [Online]. Available: https://fbullo.github.io/lrpk/